

KONWOLUCYJNE SIECI NEURONOWE

ORAZ PROFESJONALNY WORKFLOW INŻYNIERA AI

TECHNOLOGIE, NARZĘDZIA I WORKFLOW INŻYNIERA AI

Celem przedmiotu nie jest wyłącznie poznanie algorytmów AI/ML i uruchamianie kolejnych modeli na danych. Równie ważne jest nauczenie się, jak budować uporządkowane, rozwijalne i powtarzalne projekty, które da się uruchamiać, testować, analizować i rozszerzać także po zakończeniu jednego ćwiczenia czy laboratorium. W praktyce inżynieria uczenia maszynowego to nie tylko model, ale również konfiguracja eksperymentów, przygotowanie danych, automatyzacja uruchomień, kontrola jakości kodu, raportowanie wyników i praca z repozytorium, które z czasem staje się coraz większe.

Dlatego w tym zadaniu pracujecie nie tylko z kodem implementującym model, ale z całym szkieletem projektu ML. Waszym celem jest zrozumienie architektury repozytorium, sposobu uruchamiania pipeline'u, roli konfiguracji, testów i narzędzi inżynierskich oraz nauczenie się, jak rozwijać taki projekt w kontrolowany sposób. Chodzi o to, aby przejść od podejścia "piszę pojedynczy skrypt do zadania" do podejścia "rozwijam projekt, który da się utrzymać, rozszerzać i analizować".

W realizacji tych celów pomagają konkretne technologie i narzędzia, z którymi będziecie pracować w trakcie zadania. Poznacie między innymi uv do zarządzania środowiskiem i zależnościami, Makefile do automatyzacji typowych kroków pracy, pytest do testów automatycznych, ruff do lintingu i formatowania kodu, pyright do statycznej analizy typów, a także pre-commit do automatycznego sprawdzania jakości zmian przed commitem lub wypchnięciem kodu. Zobaczycie również, jak wykorzystywać pliki konfiguracyjne YAML, walidację konfiguracji przez Pydantic, interfejsy CLI uruchamiane z terminala oraz strukturę projektu podzieloną na warstwy odpowiedzialne za dane, eksperymenty, trening, ewaluację i raportowanie. Dzięki temu nauczycie się nie tylko "trenować model", ale też pracować w sposób bardziej zbliżony do rzeczywistej praktyki inżynierskiej.

Repozytorium, na którym pracujemy, znajduje się pod adresem <https://gitlab.com/gmadejski-ug/ml-project-template>. Należy je sforkować do własnego konta, a następnie pracować na swoim forku. Gałąź main zawiera podstawową wersję projektu, natomiast gałąź mlflow zawiera wariant rozszerzony o integrację z narzędziem MLflow. W zależności od etapu zajęć lub zakresu zadania będziecie pracować z jedną z tych wersji.

ZADANIE 1: PROFESJONALY WORKFLOW

Skorzystaj z forka repozytoriumna gitlabie: <https://gitlab.com/gmadejski-ug/ml-project-template>

Następnie wykonaj następujące zadania. Jeśli zadanie prosi o odpowiedź na pytanie, lub notatkę, możesz je wykonać w tym repozytorium w formie dodatkowego pliku markdown (obok readme.md).

a) Narysuj architekturę projektu

Zadanie: Narysuj schemat blokowy, jak rozumiesz architekturę tego projektu, np. w draw.io, albo innym narzędziu.

Zaznacz przepływ fetch -> preprocess -> train -> evaluate -> visualize -> compare. Wskaż też, które katalogi pełnią rolę wejścia, a które wyjścia. Zapisz plik ze schematem do repozytorium.

Cel: Zrozumieć rolę configs/, src/cli/, src/experiments/, data/, models/, reports/, tests/.

b) Uruchom projekt z terminala

Zadanie: Uruchom make setup, make pipeline, make test, make pyright, make ruff. Następnie opisz własnymi słowami, co sprawdza każde z tych poleceń i po co jest potrzebne w projekcie ML.

Cel: Ośwoić się z uv, Makefile, testami, lintingiem i type-checkingiem.

c) Przeczytaj konfigurację eksperymentów

Zadanie: Otwórz pliki YAML w katalogu configs/ i porównaj je. Wypisz, czym różnią się gotowe eksperymenty i które pola konfiguracji wpływają na model, preprocessing oraz ścieżki zapisu artefaktów.

Cel: Zrozumieć, jak projekt jest sterowany konfiguracją zamiast ręcznego przepisywania kodu.

d) Prześledź wykonanie jednego eksperymentu

Zadanie: Uruchom jeden eksperyment i prześledź w kodzie, co dzieje się od wejścia przez `src/cli/run_experiment.py` aż do zapisania wyników w `reports/` i `models/`. Przygotuj krótką notatkę: które funkcje są wywoływane i w jakiej kolejności.
Cel: Zobaczyć, jak działa CLI i jak kod jest podzielony na warstwy.

e) Wyjaśnij klasę bazową eksperymentu

Zadanie: Przeczytaj `src/experiments/base.py` i opisz, które elementy są wspólne dla wszystkich eksperymentów, a które mogą się zmieniać.
Cel: Zrozumieć ideę klasy bazowej, wspólnego pipeline'u i rozszerzania projektu przez nowe eksperymenty.

f) Sprawdź, jak działa walidacja konfiguracji

Zadanie: Przeczytaj `src/utlils/config.py`. Następnie przygotuj 2 błędne pliki YAML, które powinny zostać odrzucone przez walidację, i opisz dlaczego.
Cel: Zrozumieć rolę Pydantic i sens walidowania konfiguracji przed uruchomieniem eksperymentu.

g) Dodaj nowy wariant eksperymentu bez zmiany kodu

Zadanie: Skopiuj jeden z istniejących plików konfiguracyjnych i utwórz nowy eksperyment, zmieniając np. `epochs`, `learning_rate` albo `hidden_dims`. Uruchom go i porównaj wyniki z eksperymentem bazowym.
Cel: Nauczyć się rozszerzać projekt najpierw przez konfigurację.

h) Dodaj nowy preprocessing

Zadanie: Rozszerz projekt o nową strategię preprocessingu, np. `robust`. Dodaj odpowiednią obsługę w kodzie, zaktualizuj typy/walidację konfiguracji i dopisz testy.
Cel: Przećwiczyć modyfikację kodu produkcyjnego razem z testami.

i) Dodaj nowy model albo nowy typ eksperymentu

Zadanie: Dodaj prosty nowy wariant modelu klasyfikacji i zarejestruj go tak, aby dało się go uruchomić przez konfigurację. Przygotuj dla niego osobny plik YAML.
Cel: Zobaczyć, jak działa rejestr eksperymentów i jak rozwija się architekturę bez psucia istniejącego kodu.

j) Rozszerz raportowanie wyników

Zadanie: Dodaj do raportów nową metrykę albo nowy wykres. Upewnij się, że wynik zapisuje się do `reports/` i da się go porównać między eksperymentami.
Cel: Zrozumieć, że projekt ML kończy się nie tylko modelem, ale też sensownym raportem.

k) Dopisz test błędu

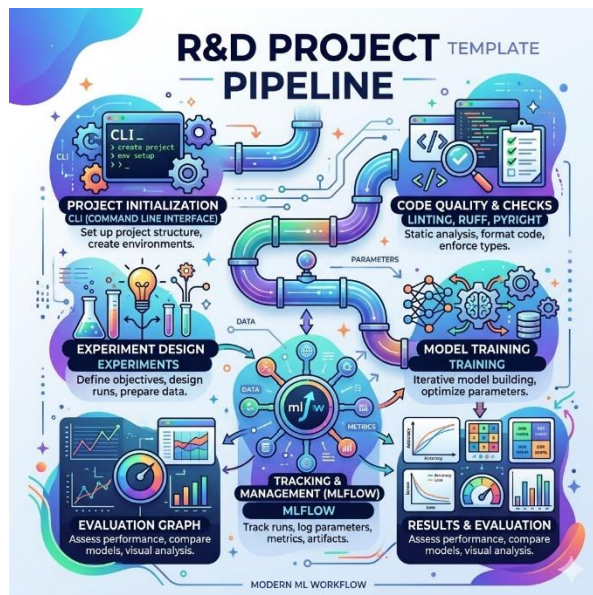
Zadanie: Dodaj test sprawdzający, co dzieje się wtedy, gdy użytkownik poda niepoprawną nazwę eksperymentu albo błędną konfigurację.
Cel: Nauczyć się pisać testy nie tylko dla poprawnych przypadków, ale też dla błędów i sytuacji brzegowych.

l) Dodaj własne polecenie CLI

Zadanie: Dodaj nową komendę CLI, np. do podsumowania wyników eksperymentów albo wypisania dostępnych konfiguracji.
Cel: Przećwiczyć rozwijanie interfejsu terminalowego projektu.

m) Podmień dataset na `diagnosis.csv`

Zadanie: Zastąp Iris csvką z poprzednich zajęć `diagnosis.csv` prostym zbiorem tabelarycznym i opisz, które elementy projektu trzeba było zmienić, a które pozostały bez zmian.
Cel: Sprawdzić, czy architektura projektu rzeczywiście oddziela elementy ogólne od specyficznych dla danych.

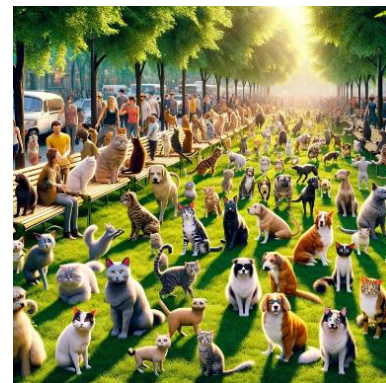


ZADANIE 2: CNN DLA PSÓW i KOTÓW

Uwaga, dla chętnych: to narzędzie można (ale nie trzeba) przeprowadzić na bazie szablonu z zadania 1. Byłoby to bardzo pożyteczne, ale pewnie będzie czasochłonne. W dodatku a gałęzi „mlflow” udostępnionego repo jest fajne narzędzie do zachowywania wyników i modeli na gitlabie. Możecie spróbować je wykorzystać.

Celem zadania jest wykorzystanie sieci konwolucyjnych w bibliotece PyTorch do rozpoznawania psów i kotów na obrazach.

Paczka ze zdjęciami (miniaturkami) znajduje się w załączniku (dogs-cats-mini.zip) i zawiera dokładnie 12 500 zdjęć kotów oraz 12 500 zdjęć psów (łącznie około 26 MB). Podpunkty do realizacji:



a) Zapoznaj się z wybranym samouczkiem lub przykładem dotyczącym klasyfikacji obrazów w PyTorch, np.:

- oficjalny tutorial PyTorch dotyczący transfer learningu w klasyfikacji obrazów: https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html
- przykładowy notebook Kaggle: <https://www.kaggle.com/code/tirendazacademy/cats-dogs-classification-with-pytorch>
- inny materiał oparty na PyTorch dotyczący klasyfikacji obrazów psów i kotów.

b) Załaduj zbiór danych i przeprowadź jego wstępne przetwarzanie, w szczególności:

- rozpakowanie i wczytanie obrazów,
- wyznaczenie etykiety klasy (cat / dog) na podstawie nazwy pliku,
- przygotowanie zbiorów: treningowego, walidacyjnego (i ewentualnie testowego),
- wykonanie transformacji obrazów w PyTorch, np. zmiana rozmiaru, konwersja do tensora, normalizacja oraz — opcjonalnie — augmentacja danych.

c) Zbuduj i wytrenuj model konwolucyjnej sieci neuronowej (CNN) na zbiorze treningowym.

Architektura może być standardowa, np. oparta na kilku warstwach Conv2d, ReLU, MaxPool2d, a następnie warstwach w pełni połączonych. Implementacja powinna być wykonana w PyTorch.

d) Dokonaj walidacji modelu.

Przedstaw krzywe uczenia dla zbioru treningowego i walidacyjnego, np. przebieg funkcji straty i dokładności w kolejnych epokach.

Pytania:

- Czy w zbiorze występują koty przypominające psy albo psy przypominające koty?
- Ile takich przypadków można wskazać?
- Czy potrafisz znaleźć i pokazać konkretne obrazy błędnie sklasyfikowane przez model?

e) Powtórz eksperyment z punktów c–d dla kilku różnych konfiguracji sieci, zmieniając np.:

- optymalizator (SGD, Adam),
- funkcję aktywacji,
- liczbę warstw,
- liczbę filtrów,
- zastosowanie dropout,
- parametry augmentacji danych.

Wskaż, która konfiguracja działała najlepiej, i przedstaw jej wyniki, w szczególności:

- krzywe uczenia,
- końcową dokładność,
- macierz pomyłek.

W razie dłuższego treningu pamiętaj o możliwości zapisywania modelu do pliku i późniejszego wznowienia treningu.

f) Pytanie: czy można wykorzystać model pre-trained, a następnie go dotrenować, aby uzyskać lepsze rozpoznawanie psów i kotów?
Przeprowadź eksperyment z użyciem transfer learningu w PyTorch, np. z modelem takim jak ResNet, MobileNet albo EfficientNet, i porównaj jego wyniki z modelem trenowanym od zera. Oficjalny tutorial PyTorch pokazuje dokładnie taki schemat pracy dla klasyfikacji obrazów.